

OOPS Fundamentals in JAVA

Monday, December 23, 2024 3:39 PM

- Overview of OOPS
- Objects and classes
- Pillars of OOPs
 - Data abstraction
 - Encapsulation
 - Inheritance
 - Types of inheritance (single, multiple, hybrid)
 - Polymorphism
 - Static poly
 - Dynamic poly
- Relationship
 - Is-a relationship
 - Has-a relationship

OOPS means Object Oriented Programming Language
Here object means real world entity like Car, Bike, ATM...etc

Procedural Programming (focus on functions not data)	OOPS (focus on data)
Program is divided into parts called functions	Program is divided into Objects
Does not provide proper way to hide data, gives importance to function and data moves freely.	Objects provide data hiding, gives importance to data
Overloading is not possible	Overloading is possible
Inheritance is not possible	Inheritance is possible
Code reusability doesn't present	Code reusability is present
Example: Pascal, C, FORTRAN etc.	Examples: Java, C#, Python, C++ etc.

Object and Class:

Object has 2 things - Properties or state, Behaviour or function

Ex: Dog is an object because it has -

- Properties like : Age, breed, color..etc
- Behaviours like Bark, Sleep, Eat...etc

Car is an Object because it has:

- Properties : Color, Type, Brand, weight...etc
- Behaviour: apply break, drive, increase speed...etc

To Create an Object a class is required.

- So, Class provides the template or blueprint from which Object can be created.
- From one Class, we can create multiple Objects.
- To create a class, use the keyword 'class'.

Example:

```
class Student{
```

```
    int age;
```

```
    String address;
```

```
    String name;
```

```
    double phonenum;
```

```
#all these(properties) are known as data variables
```

```
Update Address(){  
#data method  
}  
Display name(){  
#data method  
}  
  
}  
Student obj1 = new Student();  
#like these we can create as many as objects we want
```

1st Pillar of OOPS - Data Abstraction

- It hides the internal implementation and shows only essential functionality to the user.
- It can be achieved through interface and abstract classes.

Example:

- Car - we only shown the BREAK pedal, and if we press it, Car speed will reduce. But HOW? That is ABSTRACTED to us.
- Cellphone - how comm is made that is Abstracted to us.

Advantages of Abstraction:

Increased security and confidentiality.

2nd Pillar of OOPS - Data Encapsulation

Encapsulation bundles the data and the code working on that data in a single unit.

Also known as Data-hiding.

Steps to achieve encapsulation:

- Declare variable of a class as private
- Provide a public Getters and Setters to modify and view the value of the variables.

Advantages:

- Loosely coupled code
- Better access control and security

3rd Pillar of OOPS - Inheritance

It can inherit both functions and variables, so we do not have to write them again in child classes.

Can be achieved using extends keyword or thorough interface.

Types of inheritance:

- Single
- Multilevel
- Hierarchial
- Multiple (through interface we can solve diamond problem)

Advantages:

Introduction

Monday, December 23, 2024

3:39 PM

- Java Components (JVM, JRE, JDK)
- Java Data types
 - Primitive type
 - Integral type : Char, byte, short, int, long
 - Fractional type: float, double

Topics

Thursday, January 16, 2025 2:19 PM

S.no	Topics
1.	Introduction to Spring Boot? • What problem it solves? • Its features and advantages
2.	Setting up initial Spring Boot Project • Understand Layered Architecture
3.	Understand Maven pom.xml or Gradle structure a bit
4.	Spring Boot Annotations • @SpringBootApplication • @Controller • @RestController • @RequestMapping • @GetMapping, @PostMapping, @PutMapping • @DeleteMapping • @Autowired • @Service • @Repository • @ComponentScan • @Configuration • @Value • @Qualifier • @Profile • @EnableAutoConfiguration • @Entity • @Transactional • @EnableCaching • @Async • @EnableScheduling ...etc
5.	Understand about Dependency Injection and Beans
6.	Spring Boot Data Access • Spring JPA • Spring JDBC • Query Method
7.	Restful APIs with Spring Boot
8.	Spring Boot Security • Authentication and Authorization
9.	Spring Boot Logging
10.	Spring Boot Exception Handling
11.	Spring Boot Caching
12.	Spring Boot Interceptor
13.	Spring Boot Scheduling
14.	Spring Boot Unit Testing
15.	Microservices topics with Spring Boot • Introduction to microservices • Service discovery using Eureka ◦ Microservice Registering • Tracing the request through multiple microservices

	○ Sleuth and Zipkin ○ Sleuth we can attach the trace id with request ○ Zipkin helps to visualize the trace • Spring Boot profiles ○ Different properties based on environment (QA, Prod, Dev...) • Spring Cloud Config Server ○ It's a config management • Communication between different microservices ○ Sync ○ Async (messaging service) • API Gateway • Circuit Breaker • CQRS (Command Query Responsibility Segregation) • Deployment and Containerization ○ Creating executable JARs and WARs ○ Deploying to docker container
16.	Build Project

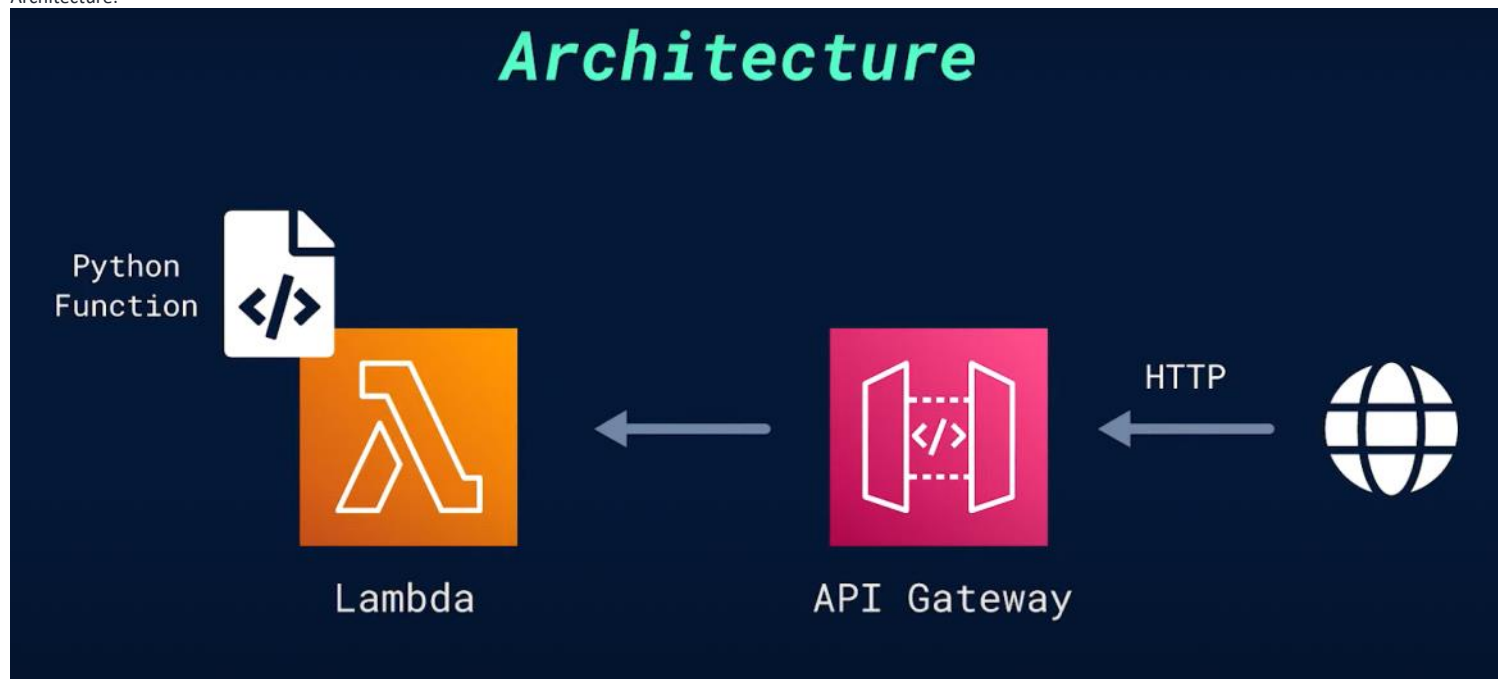
Introduction to Spring Boot

Thursday, January 16, 2025 3:34 PM

AWS Lambda

Sunday, April 13, 2025 6:54 PM

Architecture:



Sample program:

```
lambda_function x Execution results x (+)
1 import json
2 import random
3
4
5 def random_drink():
6     drinks = ["coffee", "tea", "beer", "wine", "water", "juice"]
7     return random.choice(drinks)
8
9
10 def lambda_handler(event, context):
11     drink = random_drink()
12     message = f"You should drink some {drink}"
13
14     return {
15         'statusCode': 200,
16         'body': json.dumps(message)
17     }
18
```

Test results:

Go Tools Window **Test** Deploy **Changes deployed**

lambda_function.x Execution result: x

Execution results Status: **Succeeded** Max memory used: 51 MB Time: 1.00

Test Event Name
Hello

Response
{
 "statusCode": 200,
 "body": "\"You should drink some wine\""
}

Function Logs
START RequestId: 363ae47b-edbe-4860-b20f-b39b7a93c9b8 Version: \$LATEST
END RequestId: 363ae47b-edbe-4860-b20f-b39b7a93c9b8
REPORT RequestId: 363ae47b-edbe-4860-b20f-b39b7a93c9b8 Duration: 1.00 ms Billed Duration: 2 ms Memory Size: 128 MB Max Memory Use

Request ID
363ae47b-edbe-4860-b20f-b39b7a93c9b8

With message:

Tools Window **Test** Deploy **Changes deployed** Your changes ha

lambda_function.x Execution results x

```

1 import json
2 import random
3
4
5 def random_drink():
6     drinks = ["coffee", "tea", "beer", "wine", "water", "juice"]
7     return random.choice(drinks)
8
9
10 def lambda_handler(event, context):
11     drink = random_drink()
12     message = f"You should drink some {drink}"
13
14     return {
15         'statusCode': 200,
16         'body': json.dumps({"message": message, "drink": drink})
17     }
18

```

Test results:

Go Tools Window **Test** Deploy **Changes deployed**

lambda_function.x Execution result: x

Execution results Status: **Succeeded** Max memory used: 51 MB Time: 1.41 n

Test Event Name
Hello

Response
{
 "statusCode": 200,
 "body": "{\"message\": \"You should drink some coffee\", \"drink\": \"coffee\"}"
}

Function Logs
START RequestId: d2dc3761-da1c-4f2c-82fb-a8afc7f1f5ef Version: \$LATEST
END RequestId: d2dc3761-da1c-4f2c-82fb-a8afc7f1f5ef
REPORT RequestId: d2dc3761-da1c-4f2c-82fb-a8afc7f1f5ef Duration: 1.41 ms Billed Duration: 2 ms Memory Size: 128 MB Max Memory Use

Request ID
d2dc3761-da1c-4f2c-82fb-a8afc7f1f5ef

We don't really have a way to interact with it from the outside, so the way to do is we're going to add something called as "Trigger"

Trigger - which is just a way of Lambda knowing when it should execute this function

For "Trigger", needs to use an HTTP endpoint so that we can call it from anywhere through the internet access (that could be from terminal or even the browser)

Add "API gateway" - HTTP / REST

HTTP - 71% cheaper compared to REST API

AWS Lambda (Python):

Benefits of Lambda:

- No need of managing servers
- Automatic scaling
- Pay-as-you-go pricing
- Optimization performance

Example: String-reversal

Goto Lambda (in aws console) -> functions -> create function

Lambda > Functions > Create function

Create function

Choose one of the following options to create your function.

☒ Author from scratch

Start with a simple Hello World example.

☐ Use a blueprint

Build a Lambda application from sample code and configuration presets for common use cases.

☐ Container image

Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

string-reversal

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.13

Architecture

Choose the instruction set architecture you want for your function code.

☒ x86_64

☐ arm64

Permissions

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

Change default execution role

Additional Configurations

Use additional configurations to set up code signing, function URL, tags, and Amazon VPC access for your function.

New Section 1 Page 9

Basics

Sunday, April 13, 2025 8:02 PM

1. What is AWS Lambda?
2. Why use AWS Lambda?
3. How does it work?
4. Key components
5. Common use cases
6. Important Python code concepts
7. Hands-on example

🌟 1. What is AWS Lambda?

AWS Lambda is a **serverless** compute service from Amazon Web Services (AWS). It lets you run your code without provisioning or managing servers. You just write your function (e.g., in Python), upload it, and AWS takes care of the rest—scaling, availability, and infrastructure.

Think of it as: “I only care about my code logic, not servers or maintenance.”

💡 2. Why use AWS Lambda?

- **No servers to manage** (serverless)
- **Auto-scaling** (handles from 1 to thousands of requests automatically)
- **Pay per use** (you pay only when your code runs)
- **Integrates with many AWS services** (S3, DynamoDB, API Gateway, etc.)
- **Quick to deploy and update**

⚙️ 3. How does it work?

1. You write a **function** in Python (or another supported language).
2. You **deploy** it to AWS Lambda.
3. You **trigger** the function (manually or via event: API call, file upload to S3, etc.).
4. Lambda executes the code, gives a response, and shuts down automatically.

🔗 4. Key Components of AWS Lambda (especially for Python)

Component	Description
Handler	Entry point of your Lambda function (e.g. <code>lambda_handler(event, context)</code>)
Event	Input data passed when function is triggered
Context	AWS-provided object with info about invocation, function, memory, etc.
IAM Role	Permission set allowing Lambda to access other AWS services
Trigger	What causes the Lambda to run (API Gateway, S3 event, DynamoDB stream, etc.)
Runtime	Programming language/environment (e.g. Python 3.9)
Layers	Share libraries/code across functions
Environment Variables	Useful for storing configuration/secrets

🚀 5. Common Use Cases

- **API backend** using API Gateway + Lambda
- **Image processing** (triggered by S3 uploads)
- **Data transformation** (from DynamoDB, Kinesis, etc.)
- **Cron jobs** (scheduled events via CloudWatch)
- **Chatbots, notifications, and integrations**

🔒 6. Important Python Concepts in Lambda

Lambda expects your code to follow a **specific structure**.

☑️ Sample Python Lambda function:

```
def lambda_handler(event, context):  
    # Log the event  
    print("Received event:", event)  
  
    # Process and return  
    return {  
        'statusCode': 200,  
        'body': 'Hello from Lambda!'  
    }
```

```
}
```

- ◇ event = input data (like from API, S3, etc.)
- ◇ context = runtime info (like memory limit, request ID, etc.)

🔧 Best Practices:

- Keep functions small and single-purpose
- Use environment variables for configs
- Handle errors and add logging (print() goes to CloudWatch)
- Avoid large packages if not needed (affects cold start time)
- Package dependencies carefully if you use libraries not in standard Python

🏠 7. Mini Example – Trigger Lambda via API Gateway

Step-by-step flow:

1. Write a Python function like above
2. Deploy to Lambda
3. Create an API Gateway trigger
4. Hit the API URL from Postman or a browser
5. Your Lambda function runs and returns a response

🔑 IAM Role Example (Permissions)

To allow Lambda to access S3:

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::your-bucket-name/*"
}
```

Attach this to the Lambda's execution role.

Basics of JSON:

📖 What is JSON?

JSON stands for **JavaScript Object Notation**.

It's a lightweight **data format** used to store and exchange data. It's easy for both humans and machines to read and write.

📦 Think of JSON like:

```
{
  "name": "Alice",
  "age": 25,
  "skills": ["Python", "AWS", "Lambda"]
}
```

It's kind of like a Python dictionary, but in **text format**.

📁 Python and JSON

In Python, you use the built-in json module to **work with JSON data**.

📖 json.dumps()

⚡ What it does:

Converts a Python object into a JSON-formatted string

🔑 Example:

```
import json
data = {
  "name": "Bob",
  "age": 30,
  "is_active": True
}
json_string = json.dumps(data)
print(json_string)
```

📖 Output:

```
{"name": "Bob", "age": 30, "is_active": true}
```

Now json_string is a string that looks like JSON — ready to be sent to an API, saved to a file, or returned from a Lambda function.

📖 What about json.loads()?

While json.dumps() **converts Python → JSON string**

json.loads() does the **opposite**: JSON string → Python

```
json_data = '{"name": "Alice", "age": 25}'
```

```
python_obj = json.loads(json_data)
print(python_obj["name"]) # Alice
```

Use Case in Lambda:

When returning from Lambda (like API Gateway), you often do:

```
import json
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "body": json.dumps({"message": "Success!"})
    }
```

Because body must be a string — and `json.dumps()` turns your dictionary into a valid JSON string.

Summary

Function	Purpose
<code>json.dumps()</code>	Python  JSON string
<code>json.loads()</code>	JSON string  Python object

What You'll Build

- **Trigger:** Image uploaded to an S3 bucket
- **Lambda function:** Resizes the image using Python & Pillow
- **Output:** Resized image stored in another S3 folder

Prerequisites

1. AWS account
2. S3 buckets created:
 - your-input-bucket
 - your-output-bucket (or a folder like your-input-bucket/resized/)
3. IAM role for Lambda with access to S3
4. Python 3.9 or 3.10 selected as Lambda runtime

Step-by-Step Breakdown

Step 1: Install Pillow (Python image library)

AWS Lambda doesn't come with **Pillow**, so you'll need to include it manually.

On your local machine:

```
mkdir image-resize-lambda
```

```
cd image-resize-lambda
```

```
pip install pillow -t .
```

Create a file `lambda_function.py`:

Step 2: Python Lambda Code

```
import boto3
import os
from PIL import Image
from io import BytesIO
s3 = boto3.client('s3')
def lambda_handler(event, context):
    # Get the S3 bucket and file key from the event
    bucket_name = event['Records'][0]['s3']['bucket']['name']
    file_key = event['Records'][0]['s3']['object']['key']
    # Download the image from S3
    image_object = s3.get_object(Bucket=bucket_name, Key=file_key)
    image_content = image_object['Body'].read()
    # Resize the image using Pillow
```

```

with Image.open(BytesIO(image_content)) as img:
    img = img.resize((300, 300)) # Resize to 300x300 pixels
    buffer = BytesIO()
    img.save(buffer, 'JPEG')
    buffer.seek(0)
# Save resized image back to S3 (in 'resized/' folder or another bucket)
resized_key = f"resized/{os.path.basename(file_key)}"
output_bucket = bucket_name # or use another bucket
s3.put_object(Bucket=output_bucket, Key=resized_key, Body=buffer, ContentType='image/jpeg')
return {
    'statusCode': 200,
    'body': f"Image resized and saved to {output_bucket}/{resized_key}"
}

```

Step 3: Zip & Upload Code

zip -r image-resize.zip .

Upload this ZIP to your AWS Lambda function.

Step 4: Create Lambda Function

1. Go to AWS Lambda
2. Click **Create function**
3. Choose "Author from scratch"
4. Set runtime: **Python 3.9**
5. Set an IAM role with **S3 full access** (or custom access to your buckets)
6. Upload the ZIP file
7. Set the handler name: lambda_function.lambda_handler

Step 5: Add S3 Trigger

1. Under your Lambda function > Click **Add Trigger**
2. Choose **S3**
3. Select your **input bucket**
4. Event type: **PUT**
5. Prefix (optional): uploads/ (if you want to only trigger on a folder)
6. Save

Step 6: Test it!

1. Upload an image (e.g. uploads/cat.jpg) to your input S3 bucket
2. Lambda will trigger, resize the image, and upload to resized/cat.jpg

IAM Role Permissions Example:

```

{
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:PutObject"
  ],
  "Resource": [
    "arn:aws:s3:::your-input-bucket/*",
    "arn:aws:s3:::your-output-bucket/*"
  ]
}

```

Attach this to your Lambda's execution role.

Result

- Upload image  Lambda resizes it  Resized image saved in S3.
- No server involved!

Introduction

Saturday, January 18, 2025 1:23 PM

CICS - Customer Information control system

1. INTRODUCTION TO ONLINE -> CICS

2. SCREEN COMMUNICATION

1. DESIGN A BMS MAP

2. COMPILE => SETUP -> EXECUTE THE MAP -> SEE THE DESIGN

3. COBOL+CICS PGM

4. COMPILE -> SETUP -> EXECUTE THE TRAN -> SEE PROCESS

4 CLASS = . 90% THEORY - 10%

3. SCREEN -> SCREEN NAVIGATION

--> 1 SESSION -> U NEED TO ATTEND THESE CLASSES.

4. DATA COMMUNICATION

1. DB2 TABLE -> COBOL + CICS + DB2 PGM (MAINPGM-SUBPGM)

-- TRAINING RECORD/ YOUTUBE CHANNEL

2. VSAM KSDS -> COBOL + CICS + KSDS FILE

-- SAME OVER HERE

3. QUEUE -> COBOL + CICS + DB2 + TSQ -- LAST PGM

--> NOT ANY PLACE

--> COBOL + CICS + TDQ

5. HANDLE ERROR -> OTHER POINT -

=====

CICS CLASS 01:

MAINFRAME ->

1. STORE THE DATA

1. FILE SYSTEM 2. DATABASE
PS DB2

2. PROCESS THE DATA

1. BATCH SYSTEM

2. ONLINE SYSTEM

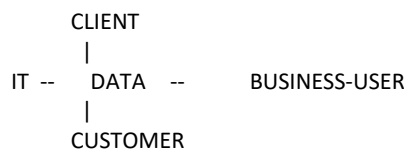
BATCH -> JCL 1. BULK DATA PROCESS 2. SCHEDULER 3. WITHOUT USER INTERACTION 4. THRU JCL

ONLINE -> CICS 1. SHORT INFORMATION 2. UNSCHEDULE 3. WITH USER INTERACTION 4. THRU CICS.

MAINFRAME APPLICATION -> ONLINE -> 1. CICS

2. IMS-DC

ONLINE SYSTEM -> QUICK ACCESS TO DATA>



I WANT TO SEE ANY DATA IN A DB2 TABLE:

1. LOGIN MAINFRAME

2. ISPF -> DB2I SCREEN

3. OPTION SPUFI -> CREATE DATASET-INPUT /OUTPUT

4. EDIT THE DATASET - CODE A SQL QUERY -> EXECUTE

5. SEE THE DATA IN SPUFI OUTPUT FILE

BUSINESS USER

DO THIS

DO THIS

MAY BE

I CANNOT

I CANNOT

CUSTOMER

NO

ONLINE APPLICATION -> USER FRONT SYSTEM

--> INTERNAL PROJECT PERSON -> MAINFRAME

CUSTOMER FRONT SYSTEM

--> EXTERNAL PERSON -> NOT INTO MAINFRAME

USER FRONT SYSTEM --> SCREEN (CICS) -> INPUT-TRIGGER -> PROCESS - PGM -> TABLE/FILE -DATA

CUSTOMER FRONT SYSTEM -. JAVA/.NET (SCREEN) - INPUT - TRIGGER -> WEB SERVICES --> TRIGGER THE CICS TRAN -> CICS -PGM -> DATA
| _____ MQ SERIES (MESSAGE QUEUE) ---|

PROJECT -> 100 APPLICATION ONLINE -> PERCENT OF HAVING -> CUSTOMER vs USER

80% - USER
20% - CUSTOMER

100 ONLINE APPLICATION 100 - CICS + 20 CUSTOMER

OUR TRAINING -> USER FRONT SYSTEM:

CICS -> CUSTOMER INFORMATION AND CONTROL SYSTEM

CICS -> DB/DC SYSTEM DB -> DATA BASE -> STORE THE DATA
DC -> DATA COMMUNICATION -> PASS THE DATA

IN OUR PROJECT -> WE USE ONLY CICS -> DC -> DATA COMMUNICATION ONLY

CICS - INTERFACE B/W APPLICATION PGM -> DATA SOURCE
IN BATCH -> PGM =EXECUTING IN Z/OS
IN ONLINE -> PGM =EXECUTED BY CICS

PGM ---> CICS --> DATA/(FILE-DB2)
|-----CICS-----|

CLASS -02:

CICS - Interface bwn Application prgm (command) - data

CICS - is a mini operating system

3 main components:

- i. Control program - task carrier
- ii. Control tables - require information of components (Developer task)
- iii. Control area - space required to do the task (system will take care)

BMS CODING:

MAPSET-NAME DFHMSD PARAMETER:

1. MAPSET NAME -> 1-7 CHAR ONLY -> WHILE EXECUTING ON THE TERMINAL -> PLACE 8TH CHAR -> TYPE.
--> DEVICE INDEPENDENT

TYPE -> - CONTROL THE OUTPUT OF THE COMPILER

TYPE=MAP --> COMPILER -> CREATE ONLY PHY MAP (LOAD)
TYPE=DSECT --> COMPILER -> CREATE ONLY SYM MAP (COPYBOOK)
TYPE=&SYSPARM --> COMPILER -> FOR BOTH - SYM + PHY
TYPE=FINAL --> END OF THE MAPSET

LANG--> USED TO TELL IN WHICH LANG - COPYBOOK TO BE CREATED..

COBOL-CICS -> LANG=COBOL
PLI - CICS -> LANG=PLI
ASM -> LANG=ASM > COBOL

MODE=INOUT

IN -> ONLY FOR INPUT -> GET THE DATA TO THE PGM
OUT -> ONLY FOR OUTPUT ->, ONLY SEND THE DATA TO SCREEN
INOUT --> BOTH

TERM=ALL , STORAGE=AUTO , MAPATTS=COLOR DEATTS=COLOR -> USE IT COPY-PASTE

TIOAPFX=YES --> BY DEFAULT =YES -

ALONG WITH UR DATA FILED -> U NEED ADDITIONAL MEMORY FOR TERMINAL I/O PROCESS
TERMINAL INPUT OUTPUT AREA PREFIX -= TIOAPFX=YES -> COMPILER 0 ADD -> 12 BYTES OF FILLER VARIABLE

12 BYTE OF FILLER VARIABLE -> 1ST VARIABLE IN THE COPYBOOK

TIOAPFX=NO -> NO MEMORY ALLOCATION./.
0> WE CANNOT DO ANY TERMINAL INTERACTION

CTRL=(FREEKB,FRSET)

FREEKB -> FREE THE KEYBOARD.
-> CICS -WHEN RECEIVES ANY INPUT -> IT WILL LOCK THE KEYBOARD
-> WHEN AM SENDING THE MAP AS RESULT -> THE KEYBOARD SHOULD BE FREE FOR INPUTS..

IF U DONT USE FREEKB -> -- X SYSTEM - LOCK THE TERMINAL

FRSET -> FIELD RESET..

ON A MAP -> I HAVE 80 FIELDS --> USER PROVIDED INPUT -> 1 FIELDS -> PRESS ENTER.

UR -PGM 1. RECEIVE ALL 80 FIELD DATA
 2. RECEIVE ONLY 1 FIELD DATA

HOW THE CICS PGM KNOWS WHICH FIELD TO RECEIVE...

1. SEND THE MAP -> RESET FIELDS -> MDT OFF (0) (MODIFIED DATA TAG)
2. USER EDITS THE FIELD -> FSET MDT ON (1)
3. CICS ALWAYS - RECEIVES ONLY THE FIELD -WHICH ARE SET (MDT ON)

NOTE: IN CASE USER -DO NOT EDIT ANY FIELDS -> ENTER -> RECEIVE NOTHING --> COMMAND FAIL -> MAPFAIL**
ANY TIME U GET ANY ERROR IN CICS COMMAND -> SYSTEM ABEND THE PROCESS. IF U DONT HANDLE THE CICS ERROR.

MSETNAME DFHMDP TYPE=&SYSPARM,LANG=COBOL,MODE=INOUT, X
 TERM=ALL,STORAGE=AUTO,MAPATTS=(COLOR), X
 TIOAPFX=YES,CTRL=(FREEKB,FRSET)

